

Complemento de Documentación API

Endpoints detectados en el frontend que NO están documentados ni implementados en el backend

Mayo 2026

3 endpoints críticos faltantes

Frontend → Backend

Contexto

Durante el análisis de comparativa entre la documentación existente y el código fuente, se identificaron **3 endpoints** que son consumidos por el frontend pero que **no existen en el backend**. Esto provoca errores 404 en tiempo de ejecución que solo se mitigan porque el frontend utiliza datos mock como fallback.

⚠ **Impacto:** Estos endpoints no están documentados en `Documentacion_API-LaEsperanza.docx` ni implementados en las rutas del backend. Sin el fallback a datos mock, la aplicación mostraría pantallas vacías o errores.

1 GET /api/v1/publicaciones/mis-publicaciones

GET /api/v1/publicaciones/mis-publicaciones **✗ NO IMPLEMENTADO**

Descripción: Devuelve todas las publicaciones creadas por el productor autenticado. Es llamado desde la página `MisPublicaciones.jsx`.

Bug #1 — ¿Por qué falla?

La ruta `GET /:id` en `publicaciones.routes.js` atrapa la solicitud con `id = "mis-publicaciones"`. Al hacer `Number("mis-publicaciones")` se obtiene `NaN`, y `prisma.publicacion.findUnique({ where: { id: NaN } })` devuelve `null`, produciendo un error 404.

Headers requeridos

Authorization

Bearer <token_jwt>

Requerido

Respuesta (200 OK)

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "titulo": "Maíz blanco primera calidad",
      "producto": { "nombre": "Maíz" },
      "precio_unitario": 120.00,
      "cantidad_disponible": 80,
      "estado": "activa",
      "negociaciones": 3,
      "created_at": "2025-04-20T00:00:00.000Z"
    }
  ]
}
```

PUT /api/v1/auth/password

PUT /api/v1/auth/password **✗ NO IMPLEMENTADO**

Descripción: Permite al usuario autenticado cambiar su contraseña. Es llamado desde la página `Perfil.jsx` en la pestaña "Seguridad".

Bug #2 — ¿Por qué falla?

La ruta `PUT /password` no existe en `auth.routes.js`. El frontend intenta llamarla al guardar el cambio de contraseña, obteniendo un error 404. El catch en el frontend muestra "Error al cambiar contraseña".

Headers requeridos

Authorization	Bearer <token_jwt>	Requerido
Content-Type	application/json	Requerido

Body

```
{
  "password_actual": "MiPassword123!",
  "password_nueva": "MiNuevoPassword456!"
}
```

Validaciones necesarias

CAMPO	REGLA
password_actual	Debe coincidir con el hash almacenado en BD
password_nueva	Mínimo 8 caracteres. Debe ser diferente a la actual
password_nueva	Se recomienda: al menos 1 mayúscula, 1 número, 1 símbolo

Respuesta (200 OK)

```
{
  "success": true,
  "message": "Contraseña actualizada correctamente"
}
```

3 GET /api/v1/dashboard/stats

GET /api/v1/dashboard/stats **✗ NO IMPLEMENTADO**

Descripción: Devuelve estadísticas del dashboard para el usuario autenticado. Es llamado desde `Dashboard.jsx`. Al fallar, usa datos mock.

Bug #3 — ¿Por qué falla?

La ruta `GET /api/v1/dashboard/stats` no está definida en ningún archivo de rutas. El frontend ejecuta `api.get('/dashboard/stats')` que falla silenciosamente, y el catch asigna datos mock con `mockStats(user?.rol)`.

Headers requeridos

Authorization

Bearer <token_jwt>

Requerido

Respuesta esperada por el frontend

Para productor:

```
{
  "success": true,
  "data": {
    "publicaciones": 8,
    "negociaciones_activas": 3,
    "entregas_pendientes": 2,
    "ingresos_mes": 12450
  }
}
```

Para comprador / asociación:

```
{
  "success": true,
  "data": {
    "publicaciones_vistas": 45,
    "negociaciones_activas": 5,
    "entregas_pendientes": 1,
    "gasto_mes": 8900
  }
}
```

Consultas SQL sugeridas para implementación

```
-- Para productor:
SELECT
  (SELECT COUNT(*) FROM publicaciones WHERE productor_id = ?) AS publicaciones,
  (SELECT COUNT(*) FROM negociaciones WHERE productor_id = ? AND estado IN ('pendiente','en_
  (SELECT COUNT(*) FROM entregas e JOIN negociaciones n ON e.negociacion_id = n.id WHERE n.p
  (SELECT COALESCE(SUM(monto), 0) FROM pagos p JOIN negociaciones n ON p.negociacion_id = n.
```

Endpoints adicionales documentados en la arquitectura pero sin ruta propia

propia

ENDPOINT	DOCUMENTADO COMO	REALIDAD	ESTADO
GET /api/v1/compradores	Listar compradores (arquitectura menciona CRUD completo)	Solo existe GET /:id y PUT /:id	✗ No implementado
Mensajes como módulo independiente	Módulo separado con su propio router	Anidado bajo /api/v1/negociaciones/:id/mensajes	✓ Funcional (anidado)
GET /api/v1/notificaciones/no-leidas	Conteo rápido de no leídas (badge en topbar)	El frontend tiene badge hardcodeado con valor "3"	✗ No implementado

Recomendación: Agregar un endpoint GET /api/v1/notificaciones/no-leidas que devuelva solo el conteo de notificaciones no leídas para el badge del topbar:

```
// Respuesta:
{ "success": true, "noLeidas": 5 }
```